

Báo cáo bài tập EX14.3: Long Short-Term Memory (LSTM)

1. Về điều chỉnh Hyperparameters

- num_hiddens (Số đơn vị ẩn): Tăng tham số này giúp mô hình thông minh hơn, học được các quy luật ngôn ngữ phức tạp nhưng sẽ làm tốc độ tính toán chậm lại đáng kể và dễ gây hiện tượng quá khớp (overfitting).
- num_steps (Độ dài chuỗi): Giúp mô hình có "tầm nhìn" xa hơn để hiểu ngữ cảnh dài (ví dụ: nhớ chủ ngữ ở đầu câu để chia động từ ở cuối câu), nhưng làm tăng thời gian huấn luyện vì phải tính toán qua nhiều bước thời gian hơn.
- lr (Tốc độ học): Không ảnh hưởng đến thời gian chạy hay độ phức tạp nhưng quyết định mô hình có hội tụ được hay không; nếu quá cao, đầu ra sẽ chỉ là những ký tự ngẫu nhiên vô nghĩa.

```
def get_lstm_params(vocab_size, num_hiddens, device):
    num_inputs = num_outputs = vocab_size

    def normal(shape):
        return torch.randn(size=shape, device=device) * 0.01

    def three():
        return (normal(
            (num_inputs, num_hiddens)), normal((num_hiddens, num_hiddens)),
            torch.zeros(num_hiddens, device=device))

    W_xi, W_hi, b_i = three() # Input gate parameters
    W_xf, W_hf, b_f = three() # Forget gate parameters
    W_xo, W_ho, b_o = three() # Output gate parameters
    W_xc, W_hc, b_c = three() # Candidate memory cell parameters
    # Output layer parameters
    W_hq = normal((num_hiddens, num_outputs))
    b_q = torch.zeros(num_outputs, device=device)
    # Attach gradients
    params = [
        W_xi, W_hi, b_i, W_xf, W_hf, b_f, W_xo, W_ho, b_o, W_xc, W_hc, b_c,
        W_hq, b_q]
    for param in params:
        param.requires_grad_(True)
    return params
```

2. Về việc thay đổi mô hình để học theo "Từ" (Word-level)

- Cần thay đổi cách Tokenization: Thay vì cắt văn bản thành từng chữ cái, ta cắt thành từng từ dựa trên khoảng trắng.
- Phải sử dụng thêm lớp Embedding: Do số lượng từ trong thực tế rất lớn (hàng chục nghìn), ta cần lớp này để nén các từ thành các vector số học nhỏ gọn trước khi đưa vào LSTM để tiết kiệm bộ nhớ.

3. Về so sánh chi phí tính toán (RNN, GRU, LSTM)

- RNN: Chi phí thấp nhất vì chỉ có 1 tập tham số, nhưng khả năng ghi nhớ kém nhất.
- GRU: Chi phí trung bình (gấp khoảng 3 lần RNN) nhờ lược bỏ bớt cổng so với LSTM nhưng vẫn giữ được khả năng nhớ dài hạn tốt.
- LSTM: Chi phí cao nhất (gấp khoảng 4 lần RNN) cả trong huấn luyện và suy luận vì có tới 4 hệ thống cổng cần tính toán song song.

4. Về lý do Hidden State cần hàm tanh

- Kiểm soát sự bùng nổ: Ô nhớ C là kết quả của các phép cộng tích lũy liên tục qua thời gian, nên giá trị của nó có thể tăng rất lớn; hàm tanh giúp nén giá trị đó về lại khoảng $[-1, 1]$.
- Đảm bảo tính ổn định: Việc ép đầu ra của trạng thái ẩn về một khoảng cố định giúp các tầng tính toán phía sau (như tầng Softmax) không bị lỗi tràn số và hoạt động ổn định hơn.